

Konstanz, 28.10.2025

Übungsblatt 1b

„Algorithmen und Datenstrukturen WS 2025/26“

Abgabe: bis 04.11.2025, 14:00, auf Moodle hochladen

Aufgabe 3 (GPU-Beschleunigung)

2 Punkte

Ist es möglich die Komplexität eines Algorithmus dadurch zu verbessern, dass man ihn auf dem Grafikprozessor (GPU) ausführt? Bitte verwenden Sie beziehen Sie sich bei Ihrer Antwort ausschließlich auf die Definitionen der Vorlesung!

Die GPU verändert nichts an der Komplexität eines Algorithmus.
Wenn man beachtet wie viele Operationen es durchführt, ist es irrelevant ob es auf einer starken CPU oder einer starken GPU läuft. Faktisch verändert sich eventuell die Laufzeit, ja, aber die Komplexität in sinne von durchgeführten Operationen ändert sich nicht und hat auch keine Auswirkungen auf die asymptotische Komplexität oder die Effizienz des Algorithmus

Aufgabe 4 (Laufzeit von Schleifen)

15 = 1+2+2+3+3+4 Punkte

Geben Sie für die folgenden Code-Fragmente die Laufzeit an **und begründen Sie ihre Antwort**. Dabei hat die Anweisung **A** den Aufwand $O(1)$ und beeinflusst die Schleifenköpfe nicht.

a)

```
for (int i = 0; i <= n; i++) { <- O(n)
    A; <- constant O(1)
}
```

wenn: $n = 5$

0,1,2,3,4,5
-> 6 Iterationen, $n+1$

Die Schleife führt A ($O(1)$) für $i = 0$ bis n aus, also $(n + 1)$ -mal.
Damit wächst die Laufzeit linear mit n $O(n)$.

$$\begin{aligned} T(n) &= O(n + 1) * O(1) \\ T(n) &= O(n + 1) \\ T(n) &= O(n) \end{aligned}$$

Regel 2:
Laufzeit innere * Anz Iterationen äußere (unabhängige schleifen)

b)

```
for (int i = 0; i <= n; i++) { O(n)
  for (int j = 0; j <= i; j++) { O(n/2)
    A;
  }
}
```

innere schleife läuft abhängig der äußeren, daher hat die innere schleife $O(n/2)$

Stellen wir mal jede Ausführung als x dar:

i=0	wäre die innere schleife
i=1 X	abhängig von n, wäre es ein
i=2 XX	komplettes quadrat, da es aber ein
i=3 XXX	dreieck ist, ist es die hälfte von n
i=4 XXXX	im durchschnitt

$$T(n) = O(n * (n/2))$$

$$T(n) = O(n^2/2)$$

$$T(n) = O(n^2)$$

oder mathematisch:

bei $n = 5$
(summe der wiederholungen) / (Anzahl)
 $0,1,2,3,4 / 5 = 10/5 = 2$
Im durchschnitt läuft die innere klammer $n/2$ mal.

c)

for schleife läuft bis n, nicht n+1 oder n-1, bis n, weil i bei 1 startet

```
for (int i = 1; i <= n; i++) { 1,2,4,8,16...n-1 -> ln(n)
  for (int j = 1; j <= n; j*=2) { n-1 log basis 2
    A;
  }
}
```

Äußere schleife $O(n)$, denn es läuft nach n und addiert sich um 1 hoch bei jeder wiederholung.

Innere schleife $O(\ln(n))$, denn durch die verdoppelung verkürzt sich die Laufzeit logarithmisch.

$$T(n) = O(n) * O(\ln(n))$$

$$T(n) = O(n * \ln(n))$$

d)

```
for (int i = 0; i < n; i++) {
  for (int j = i; j < n; j*=2) {
    A;
  }
}
```

unendlich schleife!! $0*2 = 0$

Für diesen Codeabschnitt gibt es keine gültige O-Notation, da die innere Schleife nie endet. Wenn $i = 0$, wird j ebenfalls 0 gesetzt.

Da $j = 2 \rightarrow 0 * 2 = 0$ ergibt, bleibt j immer 0 und die Bedingung $j < n$ bleibt dauerhaft wahr. Somit endet die Schleife nie, und die Laufzeit ist nicht definiert (keine endliche O-Notation).

wenn $i = 3$ und die bedingung $n+3$ ist, ist es im ende effekt immernoch n wiederholungen

e)

```
for (int i = 3; i < n+3; i++) { O(n)
  for (int j = i; j <= i*i; j++) { O(i^2) -> O(n^2)
    A;
  }
}
```

i wird hier quadriert, also quasi die quadrierten wiederholungen

Die äußere Schleife läuft von $i = 3$ bis $i < n+3$, also n Iterationen
Für jedes feste i läuft die innere Schleife von $j = i$, bis $j = i^2$, also
 $(i^2 - i + 1) \sim O(i^2) \sim O(n^2)$

$$T(n) = O(n) * O(n^2)$$

$$T(n) = O(n^3)$$

schlüsselvergleiche in rot

f)

```
double pot(double x, int n) {
    if (n <= 0) return 1; O(1)
    if (n == 1) return x; O(1)
    O(log n) double p = pot(x, n/2); rekursion, n halbiert sich bei jedem au
    if (n%2) return x*p*p; // n ungerade
    else return p*p; // n gerade
}
```

n halbiert sich jedesmal -> log n

In jedem rekursionsschritt wird n halbiert, also entstehen $\log_2(n)$ aufrufe. Jeder Aufruf benötigt nur Konstante zeit (Vergleiche + Multiplikationen). Damit ergibt sich insgesamt eine Laufzeit von $O(\log n)$.

a = anz der rekursiven Aufrufe; a=1 weil: nur ein rekursiver aufruf via $\text{pot}(x, n/2)$

b = Teilungsfaktor, wird bei jedem aufruf halbiert

f(n) = Aufwand ausserhalb der Rekursion; $O(1)$, Pro aufruf 2-3 vergleiche + Multiplikation, Konstanter Aufwand

damit ergibt sich der vergleichswert:

$$\alpha = \log_b(a) = \log_2(1) = 0$$

Nach dem Master-Theorem, Fall 2:

$$T(n) = \theta(n^\alpha \log n)$$

$$T(n) = \theta(1 * \log n)$$

$$T(n) = \theta(\log n)$$

Die Laufzeit dieser rekursion wächst logarithmisch $O(\log n)$